



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Ана Попарић

Паралелно генерисање фракталних стабала: поређење Python и Rust имплементација

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - менџор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Ана Попарић	Број индекса:	SV 74/2021
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Паралелно генерисање фракталних стабала: поређење Python и Rust имплементација
--

ТЕКСТ ЗАДАТКА:

Имплементирати паралелно решење за генерисање фракталних стабала на програмским језицима Python и Rust. Упоредити понашање имплементација у контексту Амдаловог закона јаког скалирања и Густафсоновог закона слабог скалирања. Анализирати и коментарисати резултате. При изради користити препоручену праксу из области софтверског инжењерства. Детаљно документовати решење.

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА	

Редни број, РБР :		
Идентификациони број, ИБР :		
Тип документације, ТД :		Монографска документација
Тип записа, ТЗ :		Текстуални штампани материјал
Врста рада, ВР :		Дипломски - бечелор рад
Аутор, АУ :		Ана Попарић
Ментор, МН :		Др Игор Дејановић, редовни професор
Наслов рада, НР :		Паралелно генерисање фракталних стабала: поређење Python и Rust имплементација
Језик публикације, ЈП :		српски/ћирилица
Језик извода, ЈИ :		српски/енглески
Земља публиковања, ЗП :		Република Србија
Уже географско подручје, УГП :		Војводина
Година, ГО :		2026
Издавач, ИЗ :		Ауторски репринт
Место и адреса, МА :		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страница/ цитата/табела/слика/графика/прилога)		8/54/23/15/16/0/2
Научна област, НО :		Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО :		фрактално стабло, паралелизација, Python, Rust, поређење перформанси
УДК		
Чува се, ЧУ :		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН :		
Извод, ИЗ :		У раду је имплементирано и евалуирано паралелно генерисање бинарног фракталног стабла у програмским језицима Python и Rust (<i>multiprocessing</i> наспрам <i>Rayon</i>), са поређењем кроз Амдалов закон јаког и Густафсонов закон слабог скалирања, за симетрично и асиметрично стабло са преко 8 милиона грана. Python имплементација не остварује практичну скалабилност на Windows платформи (<i>spawn/pickle</i> трошак, убрзање 1,853 при $N = 8$), док Rust остварује убрзање 3,244 и прати Амдалову прогнозу до граница <i>Hyper-Threading</i> технологије, потврђујући да је приступ заснован на нитима са краћом посла значајно ефикаснији за задатке овог типа.
Датум прихватања теме, ДП :		
Датум одбране, ДО :		17.07.2026
Чланови комисије, КО :	Председник:	Др Гордана Милосављевић, редовни професор
	Члан:	Др Мирослав Зарић, редовни професор
	Члан:	
	Члан, ментор:	Др Игор Дејановић, редовни професор
		Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Ana Poparić	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	Parallel Fractal Tree Generation: A Performance Comparison of Python and Rust Implementations	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/54/23/15/16/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	fractal tree, parallelization, Python, Rust, performance comparison	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	This thesis implements and evaluates parallel fractal tree generation in Python and Rust (<i>multiprocessing</i> versus <i>Rayon</i>), comparing through Amdahl's strong-scaling law and Gustafson's weak-scaling law, for symmetric and asymmetric trees with over 8 million branches. The Python implementation fails to achieve practical scalability on the Windows platform (<i>spawn/pickle</i> overhead, speedup 1.853 at $N = 8$), while Rust achieves a speedup of 3.244 following Amdahl's prediction up to the limits of <i>Hyper-Threading</i> technology, confirming that thread-based parallelism with work-stealing is significantly more efficient for tasks of this type.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	17.07.2026	
Defended Board, DB :	President:	Gordana Milosavljević, Phd., full professor
	Member:	Miroslav Zarić, Phd., full professor
	Member:	
	Member, Mentor:	Igor Dejanović, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	Фрактално стабло	3
2.1	Фрактали	3
2.2	Рекурзија и бинарно фрактално стабло	4
2.3	Параметри и врсте фракталних стабала	4
2.4	Величина проблема	6
3	Паралелизација	7
3.1	Процеси и нити	7
3.2	Расподела оптерећења	8
4	Закони скалирања	11
4.1	Јако скалирање и Амдалов закон	11
4.2	Слабо скалирање и Густафсонов закон	12
5	Имплементација	15
5.1	Секвенцијална имплементација	15
5.2	Стратегија поделе задатака	16
5.3	Паралелна Python имплементација	17
5.4	Паралелна Rust имплементација	19
6	Експериментална евалуација	21
6.1	Тестна платформа и методологија мерења	21
6.2	Јако скалирање	23
6.3	Слабо скалирање	27
7	Дискусија	33
7.1	Поређење апсолутних перформанси	33
7.2	Јако скалирање	34
7.3	Слабо скалирање	36
7.4	Ограничења примењених модела	36
8	Закључак	39
	Списак слика	41
	Списак листинга	43
	Списак табела	45
	Списак коришћених скраћеница	47
	Списак коришћених појмова	49
	Биографија	51
	Литература	53

Глава 1

Увод

Тема рада је имплементација и евалуација паралелног генерисања бинарног фракталног стабла у програмским језицима Python и Rust, уз поређење постигнутих перформанси. Овакво поређење омогућава анализу утицаја природе програмског језика на перформансе паралелног извршавања рачунски захтевних задатака. Python и Rust одабрани су као представници два супротна приступа програмирању — Python је широко коришћен у научној заједници и нуди једноставност на рачун перформанси, а Rust ставља перформансе и безбедност меморије у први план. Фрактално стабло представља погодан проблем за ову врсту поређења — рачунски је интензивно и природно се дели на независне подзадатке. Python паралелизација остварена је уз помоћ библиотеке *multiprocessing*, а Rust посредством библиотеке *Rayon*. Стратегија паралелизације у оба језика заснива се на подели стабла на независна подстабла која се обрађују паралелно. Перформансе су евалуиране кроз мерења јаког и слабог скалирања, уз поређење са теоријским вредностима Амдаловог и Густафсоновог закона.

У раду су приказани:

- математичка основа и рачунарски модел бинарног фракталног стабла,
- теоријски оквир паралелизације: модел процеса и нити, расподела оптерећења и закони скалирања,
- секвенцијална и паралелна имплементација у програмским језицима Python и Rust,
- резултати мерења јаког и слабог скалирања за симетрично и асиметрично фрактално стабло,
- анализа уочених образаца скалирања у контексту коришћених технологија,
- закључак и правци даљег истраживања.

Глава 2

Фрактално стабло

2.1 Фрактали

Термин *фрактал* (енг. *fractal*) увео је 1975. године математичар Беноа Манделброт (*Benoît Mandelbrot*) из латинске речи *fractus*, са значењем „сломљен“ или „разломљен“. Фрактал је геометријска фигура која се може поделити на мање делове, при чему сваки од тих делова представља, макар приближно, умањену копију целокупне фигуре [1]. Наведена особина носи назив *самосличносћ* (енг. *self-similarity*) и формално се дефинише на следећи начин:

Геометријска фигура назива се самосличном уколико у њој постоји тачка са особином да свака њена околина, без обзира на величину, садржи структуру идентичну читавој фигури. Фигура је строго самослична уколико наведено својство важи у свакој њеној тачки, односно уколико се може раставити на коначан број међусобно дисјунктних подскупова од којих је сваки геометријски подударан са читавом фигуром, уз одговарајући фактор скалирања [2].

Фракталне структуре присутне су свуда у природи — од грађе дрвећа и крвних судова до атмосферских појава попут облака и муња. Класична еуклидска геометрија не може да представи оваква неправилна природна тела, због чега *фрактална геометрија* (енг. *fractal geometry*) представља неопходан математички алат за њихов опис [3]. Пример фракталне структуре у природи приказан је на слици 1.



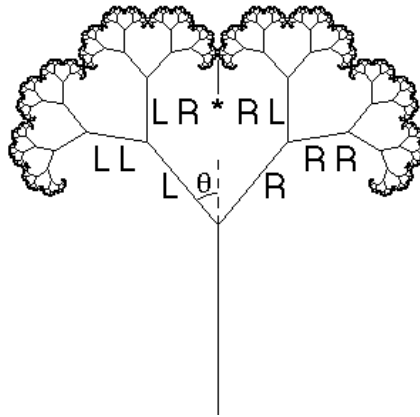
Слика 1: Фрактална структура гранања стабала у природи [4].

2.2 Рекурзија и бинарно фрактално стабло

Кључни рачунарски механизам за генерисање фракталних структура је рекурзија (енг. *recursion*) — техника решавања проблема која задатак разлаже на мање задатке исте форме, при чему функција позива сама себе са умањеним потпроблемом, све до достизања основног случаја (енг. *base case*) при коме се извршавање зауставља [5].

Бинарно стабло (енг. *binary tree*) је хијерархијска структура у којој сваки унутрашњи чвор има тачно два потомка — левог и десног [6]. Бинарно фрактално стабло комбинује ову структуру са принципом самосличности: свака грана рекурзивно се дели на две подгране које су умањене верзије родитеља, а основни случај је достизање минималне дужине гране $l_{\min}$ [7].

Генерисање стабла почиње од дебла (енг. *trunk*) дужине L_0 . Свака грана дели се на две подгране чија је дужина r пута мања од дужине родитељске гране, при чему свака подграна расте под углом θ у односу на правац родитеља. Исти поступак рекурзивно се примењује на свакој новонасталој грани, а рекурзија се зауставља када дужина гране падне испод минималног прага $l_{\min}$ — гране које достигну тај праг постају листови стабла [7]. Пример генерисаног фракталног стабла приказан је на слици 2.



Слика 2: Пример рачунарски генерисаног бинарног фракталног стабла [8].

2.3 Параметри и врсте фракталних стабала

Бинарно фрактално стабло одређено је следећим параметрима [7]:

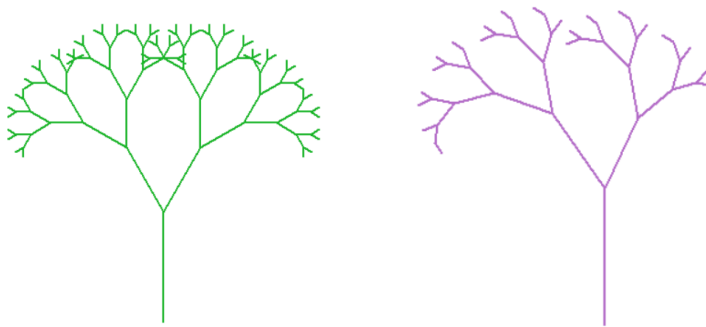
- L_0 — почетна дужина дебла,
- r — фактор смањења дужине гране у односу на родитеља ($0 < r < 1$),
- θ — угао под којим свака подграна расте од правца родитеља,
- $l_{\min}$ — минимална дужина гране испод које рекурзија престаје.

У зависности од тога да ли се исти или различити параметри примењују на леву и десну грану, разликују се два типа стабла: симетрично и асиметрично фрактално стабло.

Симетрично фрактално стабло користи исте параметре за обе подгране — исти фактор смањења и исти угао гранања, при чему обе подгране расту под истим углом у односу на родитеља, само у супротним смеровима [9]. У оквиру овог рада, симетрично стабло је дефинисано параметрима $r_l = r_d = 0.67$ и $\theta_l = \theta_d = 30^\circ$.

Асиметрично фрактално стабло уводи различите параметре за леву и десну грану. У литератури се описују два основна приступа [9]: *Shifted* стабло, код кога се асиметрија постиже померањем једне гране за одређени фактор, и стабло типа *Dragon curve*, код кога се користе два различита угла гранања. У овом раду асиметрија је остварена комбинацијом оба приступа — увођењем различитих фактора смањења ($r_l = 0.67$, $r_d = 0.57$) и различитих углова гранања ($\theta_l = 35^\circ$, $\theta_d = 25^\circ$). Оба типа стабла приказана су на слици 3.

Параметри симетричног стабла одабрани су тако да генерисана структура визуелно одговара природним биолошким стаблима — са умереним фактором смањења и широким углом гранања. Код асиметричног стабла, разлика у факторима смањења леве и десне гране намерно је наглашена како би неједнакост подзадатака била мерљива у резултатима паралелизације.



Слика 3: Симетрично (лево) и асиметрично (десно) фрактално стабло.

Код симетричног стабла, идентични фактори смањења ($r_l = r_d$) гарантују да оба подстабла имају исти број грана на свакој дубини, па је расподела посла између леве и десне рекурзивне гране савршено равномерна — идеалан случај за паралелизацију. Асиметрично стабло, напротив, генерише подстабла неједнаких величина: лева грана са $r_l = 0.67$ скалира се спорије него десна са $r_d = 0.57$, па лево подстабло генерише дубљу рекурзију и значајно већи број грана. Ово уноси неравномерност у расподелу задатака и открива способност паралелних стратегија да се носе са реалистичнијим оптерећењима. Поред тога, асиметрично стабло ближе одговара природним биоло-

шким структурама, у којима различити спољни фактори — светлост, гравитација и ветар — узрокују неравномеран раст грана [3].

2.4 Величина проблема

Максимална дубина рекурзије $d_{\max}$ одређена је условом заустављања: грана постоји све док је њена дужина $l \geq l_{\min}$. Фактор смањења r представља фактор контракције у смислу Барнслијеве теорије [10, стр. 74], па дужина гране на дубини d износи:

$$l_d = L_0 \cdot r^d$$

Из услова заустављања $L_0 \cdot r^d \geq l_{\min}$ непосредно следи максимална дубина рекурзије:

$$d_{\max} = \lfloor \log(l_{\min}/L_0) / \log(r) \rfloor$$

За параметре симетричног стабла коришћене у овом раду ($L_0 = 100$, $r = 0.67$, $l_{\min} = 0.01$) добија се:

$$d_{\max} = 22$$

За асиметрично стабло максимална дубина израчунава се одвојено за сваку грану: лева грана са $r_l = 0.67$ достиже дубину $d_l = 26$, а десна са $r_d = 0.57$ достиже дубину $d_r = 18$.

Укупан број генерисаних грана расте експоненцијално са смањењем прага $l_{\min}$. У случају симетричног стабла, када обе гране деле исти фактор скалирања, важи формула за потпуно бинарно стабло [6]:

$$N = 2^{d_{\max}+1} - 1$$

За наведене параметре добија се $N = 8\,388\,607$ грана.

За асиметрично стабло ова формула не важи, јер лева и десна страна достижу праг $l_{\min}$ на различитим дубинама ($d_l = 26$ наспрам $d_r = 18$), па генеришу различит број грана. Вредност $l_{\min} = 0.0023$ за асиметрично стабло одабрана је тако да укупан број грана буде упоредив са симетричним и износи $8\,464\,173$.

Добијени бројеви грана, преко 8 милиона у оба случаја, указују да је генерисање фракталног стабла рачунарски захтеван задатак, те да примена техника паралелног програмирања представља оправдан приступ смањењу времена извршавања.

Глава 3

Паралелизација

3.1 Процеси и нити

Процес је програм у извршавању који захтева системске ресурсе: процесорско време, меморију и улазно-излазне уређаје [11, стр. 106].

Меморијски простор процеса подељен је у четири логичке целине [11, стр. 106–107], приказане на слици 4:

- *текстуална секција* — садржи извршни код програма,
- *секција података* — чува глобалне и статичке променљиве,
- *хип* — динамички додељена меморија током извршавања,
- *стек* — локалне променљиве, параметри функција и повратне адресе.



Слика 4: Распоред меморијског простора процеса [11, стр. 107].

Оперативни систем обезбеђује потпуну изолацију меморијских простора процеса, чиме се спречава да грешка у једном процесу угрози остале [11, стр. 109]. Услед ове изолације, процеси не могу директно приступити меморији једни других, па комуникација међу њима захтева посебне механизме међупроцесне комуникације (енг. *Inter-Process Communication* — IPC) [11, стр. 123].

Поред меморијског простора, оперативни систем за сваки процес одржава блок контроле процеса (енг. *Process Control Block* — PCB), који чува бројач програма, вредности регистра и информације о додељеним ресурсима [11, стр. 109–110].

Иако изолација процеса доноси стабилност, креирање новог процеса носи значајан трошак, јер подразумева додељивање меморијског простора, иницијализацију РСВ-а и успостављање ИРС канала за комуникацију. Као одговор на овај проблем, савремени оперативни системи уводе концепт нити [11, стр. 161–162].

Нит (енг. *thread*) представља основну јединицу коришћења процесора унутар процеса. Нити унутар истог процеса деле извршни код, секцију података и хип, при чему свака нит поседује властити стек и скуп регистара [11, стр. 160–161]. Дељење меморије омогућава ефикасну комуникацију, али истовремено захтева синхронизацију приступа дељеним подацима. Насупрот процесима, нити се креирају са знатно нижим трошком јер деле постојећи меморијски простор процеса [11, стр. 160–162].

Избор између модела процеса и модела нити зависи од природе задатка [11, стр. 219]:

- *CPU-bound* — задаци који троше процесорско време на израчунавања,
- *I/O-bound* — задаци код којих се претежно чека на улазно-излазне операције.

CPU-bound задаци троше процесорско време на израчунавања, не чекајући на спољне ресурсе [11, стр. 107]. Генерисање фракталног стабла природан је пример таквог задатка, јер свака рекурзивна грана захтева искључиво аритметичке операције. Пошто између подстабала не постоји зависност података, готово цео рачунски посао може да се распореди на N процеса, па паралелизација доноси значајно убрзање.

3.2 Расподела оптерећења

Паралелно извршавање захтева расподелу посла међу доступним процесорским јединицама. Симетрично стабло природно производи равномерну расподелу посла — оба подстабла имају исту дубину рекурзије и исти број грана. Асиметрично стабло, међутим, ствара подзадатке различитих величина: лева страна ($r_l = 0.67$, дубина 26) генерише знатно више грана него десна ($r_d = 0.57$, дубина 18). Значајна разлика у величини подзатака доводи до неравномерног оптерећења, због чега неке процесорске јединице завршавају раније и чекају на остале. Постизање добрих перформанси у таквим случајевима захтева динамичку расподелу оптерећења [12, стр. 2].

Најједноставнији приступ је централни ред задатака (енг. *centralized task queue*), у коме сви процесори деле исти ред. Иако је концептуално прост, овај приступ ствара уско грло јер сви процесори конкуришу за приступ истом реду [12]. Крађа посла (енг. *work-stealing*) решава овај проблем тако што сваки процесор одржава сопствени ред задатака, а када остане без посла, сам преузима задатке од других заузетих процесора — комуникација се јавља само када је неопходна [12, стр. 3]. Захваљујући томе, неједнакост у величини подзатака асиметричног стабла не захтева претходно планирање расподеле, већ сваки процесор самостално проналази посао када претходни задатак заврши.

Механизми паралелизације одређују на који начин се посао дели међу процесорским јединицама. Теоријски оквир за процену ефикасности те поделе, метрике убрзања и модели скалирања, описани су у поглављу [4](#).

Глава 4

Закони скалирања

Паралелни системи оцењују се на основу тога колико ефикасно искоришћавају додатне процесорске јединице. За ту сврху уводе се две методологије мерења — јако и слабо скалирање, као и теоријски модели који предвиђају границе убрзања.

4.1 Јако скалирање и Амдалов закон

Јако скалирање односи се на мерење убрзања програма при повећању броја процесорских јединица уз фиксну величину проблема. Убрзање се израчунава као однос времена извршавања на једној процесорској јединици и времена извршавања на N јединица [13]:

$$S = \frac{T(1)}{T(N)}$$

Сваки програм садржи делове који се не могу паралелизовати и који одређују горњу границу убрзања без обзира на број доступних процесора. Ову границу дефинише Амдалов закон, уводећи серијски удео f — удео времена које програм проводи на деловима који се не могу паралелизовати [14]. Преостали, паралелни удео програма p представља удео времена потрошеног на делове програма који се могу извршавати паралелно, при чему важи $f + p = 1$ [14]. Убрзање при N процесора износи највише [11, стр. 164]:

$$S_{\max} = \frac{1}{f + \frac{1-f}{N}}$$

Када N тежи бесконачности, убрзање конвергира ка $1/f$ — секвенцијални удео програма пресудно ограничава укупне перформансе без обзира на број додатних процесорских јединица [11, стр. 164].

У генерисању фракталног стабла, серијски сегмент чине секвенцијална изградња почетних нивоа стабла пре покретања паралелног извршавања и спајање резултата по његовом завршетку.

На основу измереног убрзања S за дато N процењује се паралелни удео p инверзијом Амдаловог закона [15]:

$$p = \frac{N(S - 1)}{S(N - 1)}$$

4.2 Слабо скалирање и Густафсонов закон

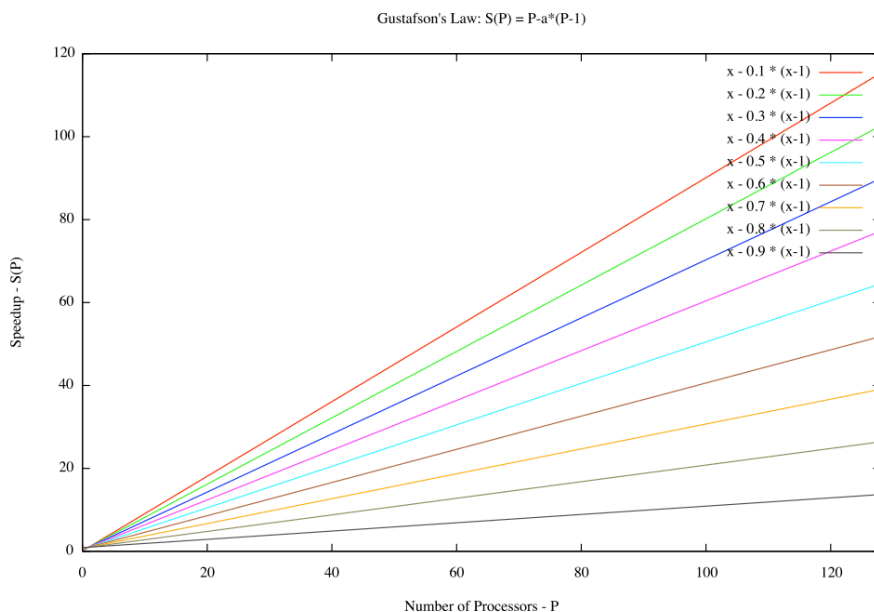
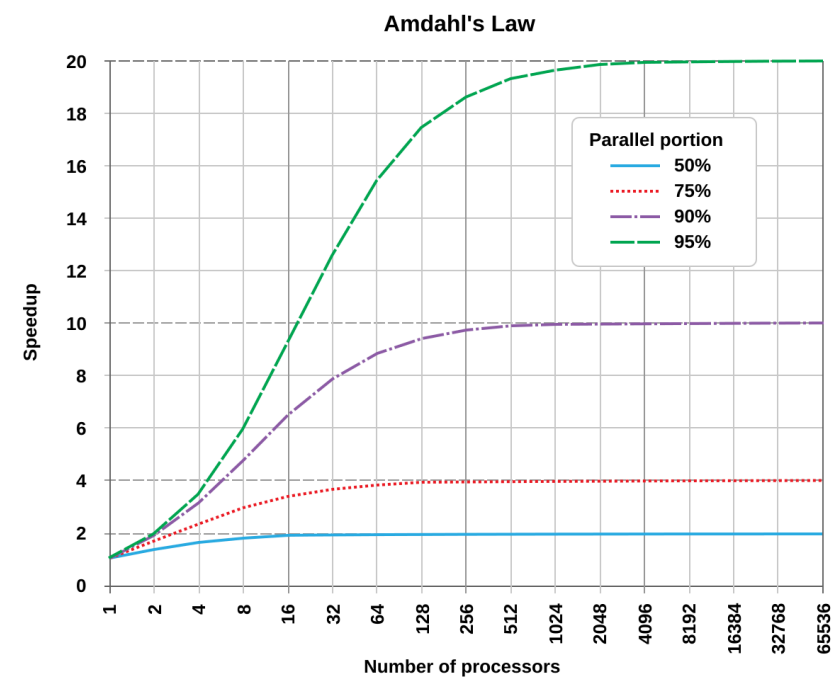
Слабо скалирање односи се на мерење при коме се величина проблема повећава сразмерно броју процесорских јединица — свака јединица задржава приближно исту количину посла. Одговарајућа мера перформанси у том режиму је количина посла обављена у фиксном времену извршавања. Скалирано убрзање представља однос хипотетичког серијског времена за проблем скалиран на N јединица и стварног времена паралелног извршавања [14]:

$$S_{\text{scaled}} = \frac{N \cdot T(1)}{T(N)}$$

Густафсонов закон полази од другачије претпоставке него Амдалов: у пракси величина проблема природно расте са бројем расположивих процесора — циљ није решити исти проблем брже, већ решити већи проблем у истом времену [14]. Кључно запажање је да паралелни сегмент програма расте пропорционално са величином проблема, док серијски сегмент остаје приближно константан. Ако са f означимо удео времена потрошеног на секвенцијални сегмент, скалирано убрзање при N процесора износи [14]:

$$S_{\text{scaled}} = N + (1 - N) \times f$$

Код Амдаловог закона убрзање расте опадајуће и асимптотски тежи вредности $1/f$, коју никад не достиже. Код Густафсоновог закона убрзање расте линеарно са бројем процесора — без фиксне горње границе, под условом да серијски сегмент остаје константан. Ова разлика приказана је на слици 5.



Слика 5: Амдалов закон (горе) — убрзање асимптотски тежи $1/f$ [16]; Густафсонов закон (доле) — убрзање расте линеарно са бројем процесора [17].

На основу измереног S_{scaled} за дато N процењује се паралелни удео p инверзијом Густафсоновог закона [14]:

$$p = \frac{S_{\text{scaled}} - 1}{N - 1}$$

Глава 5

Имплементација

У овом поглављу описана је секвенцијална и паралелна имплементација генерисања фракталног стабла у програмским језицима Python и Rust. Иако обе имплементације следе исту стратегију поделе задатака, ова два језика се значајно разликују у начину управљања меморијом и остваривања паралелног извршавања. Те разлике директно одређују избор механизма паралелизације у свакој имплементацији.

Глобална брава интерпретера (енг. *Global Interpreter Lock* — GIL) је механизам у CPython имплементацији који у сваком тренутку дозвољава извршавање Python кода само једној нити. GIL постоји јер CPython управља меморијом путем бројања референци — за сваки објекат прати се број показивача на њега, а меморија се ослобађа када тај број падне на нулу. Без заштите, истовремена измена бројача референци довела би до трке за подацима. Пошто сваки процес поседује сопствени интерпретер и меморијски простор, GIL не представља ограничење на нивоу процеса — вишепроцесно извршавање тако постаје природан избор за постизање паралелности [18].

Модел власништва (енг. *ownership*) централни је механизам програмског језика Rust којим се гарантује безбедност меморије и избегава трка за подацима у вишенитним програмима без потребе за сакупљачем смећа (енг. *garbage collector*). Правилима власништва и позајмљивања, која се проверавају у фази превођења (енг. *compile time*), грешке конкурентног програмирања постају грешке компилације, а не грешке током извршавања (енг. *runtime*) — чиме GIL као механизам заштите постаје непотребан. [19]. Библиотека *Rayon* искоришћава ове гаранције за безбедну паралелизацију преко нити [20].

5.1 Секвенцијална имплементација

Генерисање фракталног стабла засновано је на рекурзивној функцији која у сваком позиву генерише једну грану и рекурзивно обрађује леву и десну подграну. Свака грана одређена је координатама почетне тачке (x_1, y_1) и крајње тачке (x_2, y_2) , уз информацију о дубини d на којој је генерисана. Улазни параметри функције су:

- (x, y) — координате почетне тачке гране,
- l — дужина гране,
- α — угао који грана заклапа са хоризонталом.

Крајња тачка израчунава се тригонометријски:

$$x_2 = x + l \cdot \cos(\alpha), \quad y_2 = y + l \cdot \sin(\alpha)$$

Када дужина l падне испод минимума $l_{\min}$, рекурзија се прекида и грана се не генерише. У супротном, након што се текућа грана сачува, функција се позива два пута — једном за леву подграну са углом $\alpha + \theta$ и једном за десну подграну са углом $\alpha - \theta$. У оба случаја дужина гране скалира се фактором r , а крајња тачка (x_2, y_2) тренутне гране постаје почетна тачка наредног позива.

Симетрично стабло користи исте параметре r и θ за обе подгране, а асиметрично стабло уводи независне параметре — (r_l, θ_l) за леву и (r_d, θ_d) за десну подграну. Rust и Python имплементације деле ову рекурзивну логику, а разликују се у начину представљања и складиштења грана, као и у моделу паралелизације.

5.2 Стратегија поделе задатака

Будући да свака грана зависи од крајње тачке свог родитеља, алгоритам из секције 5.1 не може се директно паралелизовати. Решење лежи у подели стабла на независна подстабла. Горњи нивои стабла генеришу се секвенцијалним обилазком из секције 5.1, уз једну измену у услови заустављања — обилазак не стаје на минималној дужини гране, већ на дубини `split_depth`. Сваки чвор на тој дубини постаје корен независног подстабла и бележи се као засебан задатак описан координатама, дужином, углом и дубином. Горње гране складиште се одвојено и на крају спајају са паралелно генерисаним гранама.

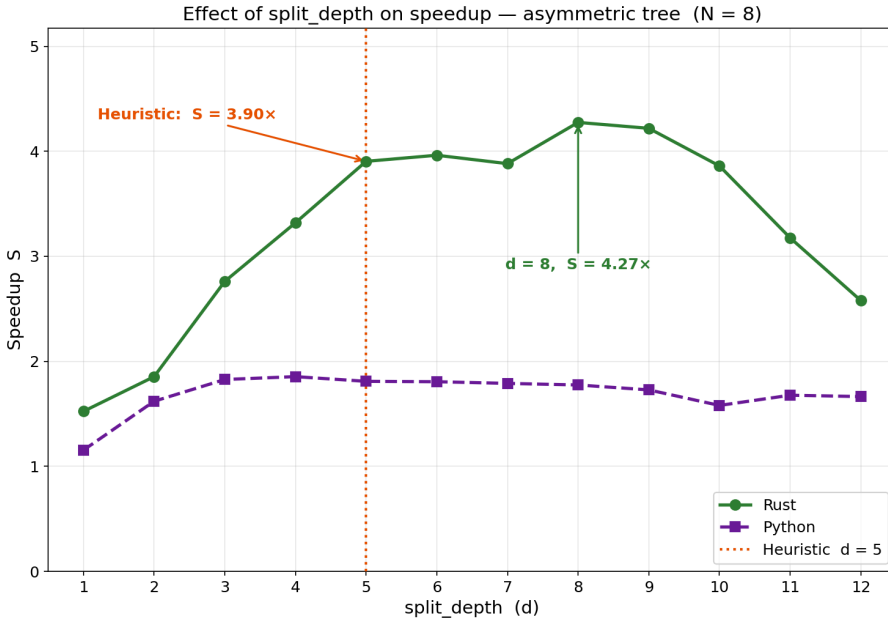
Вредност `split_depth` одређује се формулом:

$$\text{split_depth} = \lceil \log_2(N \times 4) \rceil$$

где је N број расположивих процесорских јединица. Избор фактора 4 заснован је на принципу прекомерне расподеле задатака (енг. *oversubscription*): генерише се знатно више задатака него расположивих јединица. Тиме се осигурава да јединице које заврше раније одмах преузму нове задатке, уместо да беспослено чекају. За симетрично стабло сва подстабла на дубини d имају тачно исту величину — равнотежа је увек савршена и вредност `split_depth` утиче само на секвенцијални *overhead*. За асиметрично стабло, пошто лева и десна грана расту различитим факторима смањења ($r_l \neq r_d$), подстабла на истој дубини нису исте величине — принцип *oversubscription* је у том случају важнији.

За $N = 8$ формула даје $d = \lceil \log_2(32) \rceil = 5$. Емпиријска анализа за $N = 8$ и асиметрично стабло, приказана на слици 6, потврђује да хеуристика даје добре резултате. При $d = 5$ Rust остварује убрзање $3,90\times$, док емпиријски оптимум при $d = 8$ износи $4,27\times$ — разлика од приближно 10%. У Python имплементацији убрзање је приближно константно за $d \geq 3$, јер трошак покретања процеса и серијализације резултата

доминира над неравномерношћу задатака, па вредност `split_depth` нема практичног утицаја.



Слика 6: Убрзање у зависности од `split_depth` за асиметрично стабло при $N = 8$.

Испрекидана линија означава хеуристичку вредност $d = 5$.

Хеуристика $d = 5$ представља практичан избор из два разлога. Прво, при $d = 8$ серијски се обрађује $2^8 - 1 = 255$ горњих грана пре почетка паралелног извршавања — осам пута више него при $d = 5$, што уноси значајан *overhead* трошак у серијску фазу. Друго, измерена добит од 10% специфична је за конкретну структуру стабла и хардверску конфигурацију и не може се очекивати у општем случају. Поред тога, формула $\lceil \log_2(N \times 4) \rceil$ додатно осигурава да се дубина поделе аутоматски прилагођава броју расположивих јединица — за $N = 2$ даје $d = 3$, за $N = 4$ даје $d = 4$ — чиме *overhead* трошак серијске фазе остаје у разумним границама за све вредности N .

5.3 Паралелна Python имплементација

Python имплементација остварује паралелизацију на нивоу процеса — библиотека `multiprocessing` покреће независне процесе и тако омогућава стварно паралелно извршавање на вишеструким процесорским јединицама. Комуникација главног процеса са радним процесима одвија се посредством `pickle` серијализације — задаци се пре слања пакују у бајт-ток (енг. *byte stream*), а резултати при враћању распакују [21]. Имплементација се одвија у четири фазе.

У првој фази утврђује се број радних процеса N — типично једнак броју расположивих процесорских јединица — и израчунава `split_depth` по формули из секције 5.2. Као полазна тачка рекурзије, генерише се дебло стабла засебно и уписује у листу горњих грана `upper_branches`.

У другој фази примењује се стратегија поделе описана у секцији 5.2 и производе се две листе: `upper_branches` са секвенцијално генерисаним горњим гранама и `tasks` са параметрима независних подстабала спремних за паралелну обраду.

У трећој фази задаци се дистрибуирају позивом `pool.map` над скупом процеса `Pool(processes=N)`. За сваки независан задатак позива се секвенцијална функција из секције 5.1 са одговарајућим параметрима из листе `tasks`. Пре почетка рекурзије унапред се одређује укупан број грана у подстаблу. За симетрично подстабло, рачуна се формулом $2^{d_{\max}+1} - 1$ из секције 2.4 — и алоцира се `NumPy` низ тачно потребне величине, чиме се избегава трошак реалоцирања при динамичком расту. Сваки ред низа садржи (x_1, y_1, x_2, y_2, d) . Рекурзивна функција уписује сваку грану директно на следећу слободну позицију у алоцираном низу и враћа низ грана главном процесу.

За асиметрично подстабло ова формула не важи јер лева и десна грана имају различите факторе смањења — лева страна достиже $l_{\min}$ на дубини 26, а десна на дубини 18. Број грана на некој путањи зависи искључиво од тога колико пута је путања скренула лево, а колико десно — не од редоследа тих скретања. На пример, путање лево-десно и десно-лево обе производе грану исте дужине $L_0 \cdot r_l^a \cdot r_d^b$. Функција `_count_asymmetric` искоришћава ту особину тако што стање рекурзије представља паром (a, b) , где је a број левих, а b број десних скретања од корена. Пошто многе различите путање дају исто стање (a, b) : на пример, лево-десно и десно-лево обе дају $(1, 1)$, `lru_cache` декоратор осигурава да се свако стање израчуна само једном и затим кешира — чиме се избегава поновно израчунавање.

```
@lru_cache(maxsize=None)
def count(a, b):
    if starting_length * (left_ratio ** a) * (right_ratio ** b) <
    min_length:
        return 0
    return 1 + count(a + 1, b) + count(a, b + 1)
```

Листинг 1: Мемоизована функција за бројање грана асиметричног подстабла

Стабло са преко 8 милиона грана своди се на свега 305 јединствених стања над којима функција са листинга 1 оперише. Обична рекурзија без мемоизације захтевала би позив функције за сваку грану — 8 464 173 Python позива уместо 305, што би бројање претворило у уско грло.

У четвртој фази, листа `upper_branches` претвара се у *NumPy* низ, а позивом `np.concatenate` горње гране и резултати свих паралелних процеса спајају се у јединствен низ свих грана стабла.

5.4 Паралелна Rust имплементација

Насупрот Python приступу, Rust паралелизација остварује се на нивоу нити, посредством библиотеке *Rayon*. Rust имплементација организована је по истом принципу као Python верзија из секције 5.3 — припрема, подела, паралелно извршавање и спајање резултата. *Rayon* омогућава паралелизам података тако што секвенцијалне итераторе замењује паралелним еквивалентима путем `par_iter()`, при чему логика алгоритма остаје непромењена. Захваљујући алгоритму крађе посла, *Rayon* самостално и равномерно распоређује посао међу нитима, без потребе за ручним управљањем [20] — посебно корисно када задаци нису једнаке величине, као у случају асиметричног стабла. Стратегија поделе из секције 5.2 рекурзивни проблем своди на равну листу независних задатака, над којом се паралелни итератор примењује директно.

У првој фази утврђује се број нити N позивом `thread::available_parallelism()` — еквивалентно `cpu_count()` из секције 5.3 — израчунава `split_depth` по истој формули из секције 5.2 и гради се скуп нити са N нити. Дебло стабла уписује се у вектор `upper_branches` као вредност типа `Branch` — Rust структуре са пољима (x_1, y_1, x_2, y_2, d) , еквивалентне реду у *NumPy* низу из секције 5.3.

У другој фази обилазе се секвенцијално горњи нивои стабла и за сваки чвор на дубини `split_depth` бележе се улазни параметри потребни за генерисање одговарајућег подстабла, чиме се формира вектор задатака `tasks`.

Трећа фаза обухвата паралелно извршавање задатака. Позивом

```
pool.install(|| tasks.par_iter().map(...))
```

Rayon аутоматски распоређује задатке преко скупа нити — еквивалентно `pool.map` из секције 5.3. За сваки задатак позива се секвенцијална функција из секције 5.1 са одговарајућим параметрима дефинисаним у вектору `tasks`. Пре рекурзије, одређује се укупан број грана у подстаблу и алоцира се вектор типа `Vec<Branch>` тачне величине позивом `Vec::with_capacity`.

За симетрично подстабло бројање обавља функција `count_branches` рекурзијом

$$\text{count}(l) = 1 + 2 \cdot \text{count}(l \cdot r)$$

Пошто обе подгране деле исти фактор смањења r , лево и десно подстабло увек имају исти број грана — па је довољно рекурзивно избројати само једно и помножити са 2. Функција тако прави само $d_{\max} + 1$ позива дуж једног ланца и даје исти резултат као формула $2^{d_{\max}+1} - 1$ из секције 2.4, без потребе да обилази цело стабло. Директна

примена те формуле такође би дала тачан резултат, али рекурзивни приступ је конзистентан са функцијом `count_branches_asymmetric` која следи — за асиметрично стабло аналитичка формула не постоји.

За асиметрична подстабла бројање обавља функција `count_branches_asymmetric` рекурзијом

$$\text{count}(l) = 1 + \text{count}(l \cdot r_l) + \text{count}(l \cdot r_d)$$

Пошто $r_l \neq r_d$, лево и десно подстабло имају различит број грана — рекурзија мора да обиђе оба смера и укупан број позива пропорционалан је броју грана у подстаблу, као што је приказано на листингу 2.

```
pub fn count_branches_asymmetric(
    length: f64, left_ratio: f64, right_ratio: f64, min_length: f64,
) -> usize {
    if length < min_length { return 0; }
    1 + count_branches_asymmetric(length * left_ratio, left_ratio,
    right_ratio, min_length)
    + count_branches_asymmetric(length * right_ratio, left_ratio,
    right_ratio, min_length)
}
```

Листинг 2: Рекурзивна функција за бројање грана асиметричног подстабла — Rust

Еквивалент мемоизације у програмском језику Rust захтевао би `HashMap` и прелазак на (a, b) параметризацију, што уводи додатну сложеност без реалне користи — у компајлираном коду N рекурзивних позива траје реда величине микросекунди. У програмском језику Python сваки интерпретерски позив носи значајан *overhead* трошак, па мемоизација са свега неколико стотина јединствених стања представља знатно ефикаснији приступ.

У завршној фази резултати сваке нити сакупљају се у структуру типа `Vec<Vec<Branch>>`. Будући да је за сваку нит резервисан засебан вектор, спајање резултата у јединствен низ није неопходно — укупан број грана добија се сабирањем дужина појединачних вектора.

Глава 6

Експериментална евалуација

У овом поглављу представљени су резултати мерења перформанси паралелне имплементације генерисања фракталног стабла у програмским језицима Python и Rust. Мерења обухватају јако и слабо скалирање за симетрично и асиметрично стабло, као и анализу утицаја параметра `split_depth` на ефикасност паралелизације.

6.1 Тестна платформа и методологија мерења

Сва мерења извршена су на систему чија је хардверска конфигурација приказана у табели 1. Будући да је реч о лаптоп процесору са ограниченим капацитетом хлађења, код дуготрајних експеримената може доћи до термалног пригушења (енг. *throttling*). *Throttling* је механизам којим процесор аутоматски снижава своју радну фреквенцију када температура пређе одређени праг [22].

Табела 1: Хардверска конфигурација тестног система

Атрибут	Вредност
Процесор	Intel Core i5-1035G1
Физичка / логичка језгра	4 / 8 (Hyper-Threading)
Меморија	8 GB DDR4, single-channel

Верзије коришћених компоненти дате су у табели 2.

Табела 2: Верзије коришћених компоненти

Компонента	Верзија
Оперативни систем	Windows 11 Pro
Python	3.11 (multiprocessing, spawn метода)
Rayon	1.10
rustc / cargo	1.91.0

Параметри фракталног стабла и конфигурација мерења дати су у табели 3.

Табела 3: Параметри фракталног стабла и конфигурација мерења

Параметар	Вредност
Дужина дебла	$l_0 = 100$
Фактор смањења (симетрично)	$r = 0.67$
Леви / десни фактор смањења (асиметрично)	$r_l = 0.67 / r_d = 0.57$
Угао гранања (симетрично)	$\theta = 30^\circ$
Леви / десни угао гранања (асиметрично)	$\theta_l = 35^\circ / \theta_d = 25^\circ$
Минимална дужина — симетрично (јакко скалирање)	$l_{\min} = 0.01$
Минимална дужина — асиметрично (јакко скалирање)	$l_{\min} = 0.0023$
Број понављања по конфигурацији	10
Број процесорских јединица	$N \in \{1, 2, 4, 8\}$

Вредности минималне дужине гране одабране су тако да оба типа стабла производе приближно исти укупан број грана. Симетрично стабло садржи 8 388 607, а асиметрично 8 464 173 гране, чиме је обезбеђена еквивалентна величина проблема при поређењу резултата.

Временски интервал мерења обухвата цело извршавање паралелне функције — укључујући поделу задатака и прикупљање резултата — тако да измерено време садржи и секвенцијални и паралелни део алгорита. Мерење при $N = 1$ представља чисто секвенцијалну референтну вредност, при чему се позива секвенцијална имплементација без *overhead* управљања процесима. Позивањем паралелне имплементације са $N = 1$ покренуо би се скуп процеса и извршила серијализација података, иако би сав посао био обављен само у оквиру једног процеса. Тако би цена *overhead* трошкова ушла у $T(1)$ и вештачки га увећала, па би убрзање $S(N) = \frac{T(1)}{T(N)}$ изгледало веће него што заиста јесте.

За сваку конфигурацију извршено је десет мерења ради смањења утицаја случајних варијација у извршавању. Пре израчунавања просека из скупа мерења искључене су екстремне вредности методом интерквартилног распона (IQR). Стандардна девијација σ израчунава се на основу очишћеног скупа мерења и приказана је у табелама резултата као показатељ поузданости мерења. Конфигурације су покретане у фиксном растућем редоследу ($N = 1, 2, 4, 8$), па конфигурације са већим N трче на топлијем процесору — ефекат који IQR метода делимично ублажава уклањањем екстремних вредности.

За сваку конфигурацију параметра N вредност `split_depth` одређена је формулом из секције 4.2. Табела 4 приказује конкретне вредности `split_depth` и одговарајући број задатака за сваку конфигурацију N коришћену у експериментима.

Табела 4: Вредности `split_depth` по конфигурацији; број задатака износи $2^{\text{split_depth}}$

N	<code>split_depth</code>	Број задатака
2	3	8
4	4	16
8	5	32

Исправност паралелне имплементације обезбеђена је самом структуром алгоритма поделе задатака: свако подстабло генерише се истом секвенцијалном функцијом са идентичним улазним параметрима, па се коректност резултата своди на исправност поделе. Паралелна имплементација производи исти број грана као секвенцијална за све вредности $N \in \{1, 2, 4, 8\}$, што се може верификовати из табела резултата у секцијама 6.2 и 6.3.

6.2 Јако скалирање

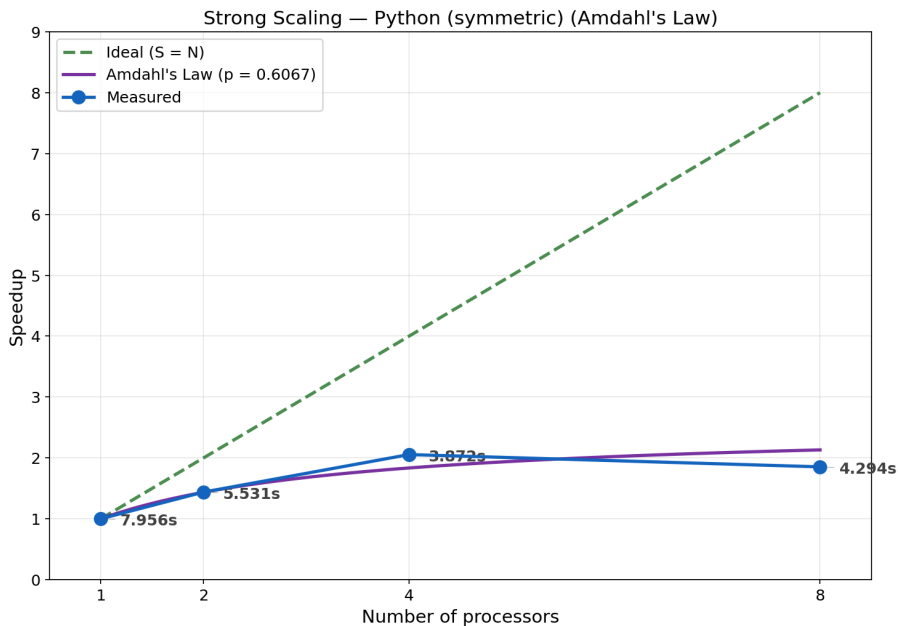
Убрзање и теоријска Амдалова крива рачунају се према дефиницијама из секције 4.1. Паралелни удео p процењује се инверзијом Амдаловог закона за свако $N > 1$, а у табелама је приказана средња вредност тих процена. Детаљна анализа уочених образаца дата је у поглављу 7.

6.2.1 Симетрично стабло

Симетрично стабло са $l_{\min} = 0.01$ садржи 8 388 607 грана. Резултати мерења приказани су у табели 5 и табели 6, а криве убрзања на слици 7 и слици 8.

Табела 5: Јако скалирање — симетрично стабло, Python ($p = 0.607$)

N	$T(N)$ (s)	σ (s)	S	Амдал
1	7.956	0.133	1.000	1.000
2	5.531	0.190	1.438	1.435
4	3.872	0.112	2.055	1.835
8	4.294	0.038	1.853	2.132

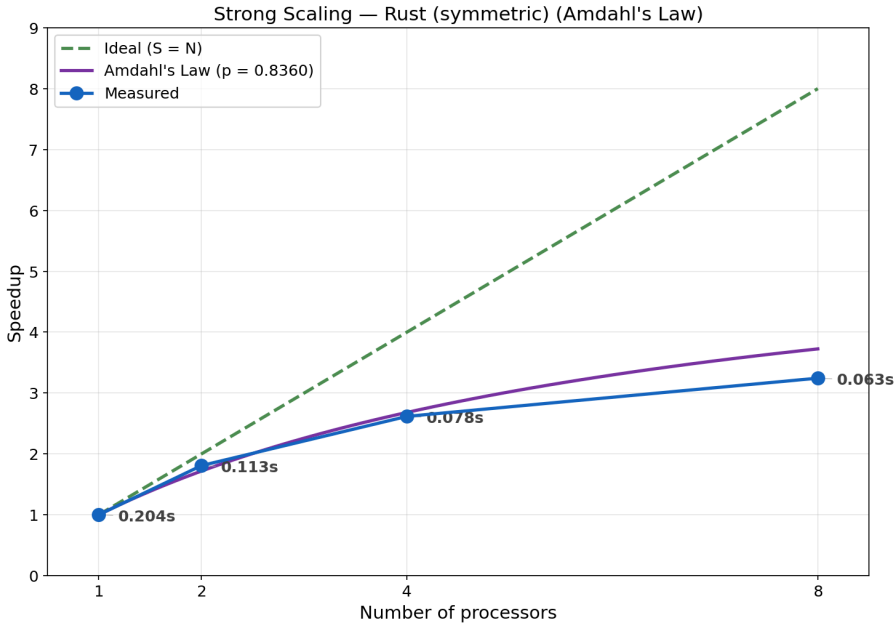


Слика 7: Убрзање при јаком скалирању — симетрично стабло, Python

Python убрзање достиже максимум при $N = 4$ ($S = 2.055$, изнад Амдалове прогнозе 1.835), а при $N = 8$ опада на $S = 1.853$ — *overhead* покретања процеса и серијализације при вишем N надмашује добит од паралелизма.

Табела 6: Јако скалирање — симетрично стабло, Rust ($p = 0.836$)

N	$T(N)$ (s)	σ (s)	S	Амдал
1	0.204	0.002	1.000	1.000
2	0.113	0.004	1.808	1.718
4	0.078	0.007	2.616	2.681
8	0.063	0.001	3.244	3.725



Слика 8: Убрзање при јаком скалирању — симетрично стабло, Rust

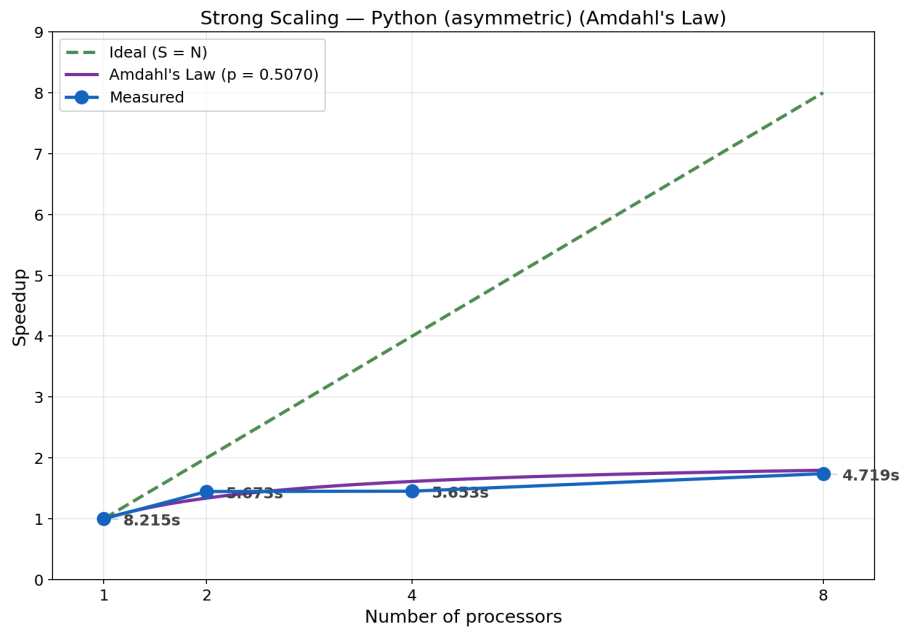
Rust убрзање монотонно расте кроз све конфигурације и при $N = 4$ готово се савршено поклапа са Амдаловом прогнозом ($S = 2.616$ наспрам 2.681). При $N = 8$ измерено убрзање заостаје за прогнозом ($S = 3.244$ наспрам 3.725) — последица *Hyper-Threading* ефекта.

6.2.2 Асиметрично стабло

Асиметрично стабло са $l_{\min} = 0.0023$ садржи 8 464 173 гране. Резултати мерења приказани су у табели 7 и табели 8, а криве убрзања на слици 9 и слици 10.

Табела 7: Јако скалирање — асиметрично стабло, Python ($p = 0.507$)

N	$T(N)$ (s)	σ (s)	S	Амдал
1	8.215	0.299	1.000	1.000
2	5.673	0.185	1.448	1.340
4	5.653	0.793	1.453	1.614
8	4.719	0.196	1.741	1.797

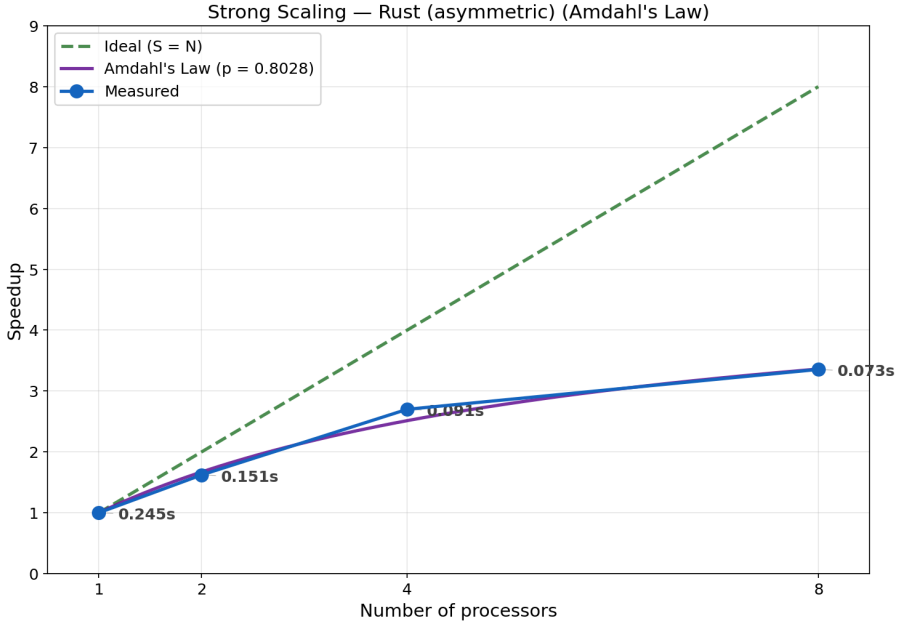


Слика 9: Убрзање при јаком скалирању — асиметрично стабло, Python

Python убрзање при $N = 4$ готово стагнира ($S = 1.453$ наспрам 1.448 при $N = 2$) — статичка расподела задатака у `pool.map` не може ефикасно да уравнотежи неједнака подстабла асиметричног стабла.

Табела 8: Јако скалирање — асиметрично стабло, Rust ($p = 0.803$)

N	$T(N)$ (s)	σ (s)	S	Амдал
1	0.245	0.012	1.000	1.000
2	0.151	0.010	1.622	1.671
4	0.091	0.004	2.700	2.513
8	0.073	0.002	3.355	3.361



Слика 10: Убрзање при јаком скалирању — асиметрично стабло, Rust

Rust убрзање монотono расте и при $N = 8$ готово савршено одговара Амдаловој прогнози ($S = 3.355$ наспрам 3.361) — боље поклапање него у симетричном случају, захваљујући алгоритму крађе посла који динамички расподељује неједнаке задатке.

6.3 Слабо скалирање

Скалирано убрзање и теоријска Густафсонова крива рачунају се према дефиницијама из секције 4.2. Паралелни удео p процењује се инверзијом Густафсоновог закона за свако $N > 1$, а у табелама је приказана средња вредност тих процена.

Величина проблема регулисана је вредношћу $l_{\min}$, која са растом N опада тако да укупни број грана расте приближно пропорционално N . Вредности $l_{\min}$ одређене су на основу структуре фракталног стабла: смањење $l_{\min}$ за фактор r додаје приближно један ниво гранања, чиме се број грана отприлике удвостручује. Пошто се N удвостручује у сваком кораку ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8$), $l_{\min}$ се множи са $r = 0.67$ за симетрично стабло, односно са $\sqrt{r_l \cdot r_d} = \sqrt{0.67 \cdot 0.57} \approx 0.618$ за асиметрично стабло. Коришћене вредности приказане су у табели 9.

Табела 9: Вредности $l_{\min}$ и број грана по конфигурацији за слабо скалирање

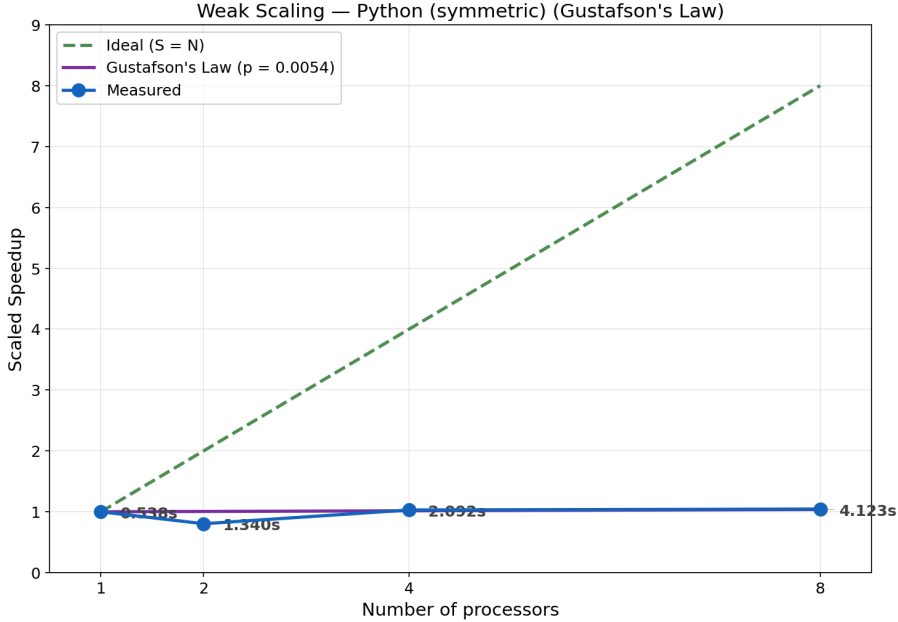
N	$l_{\min}$ (сим.)	Грана (сим.)	$l_{\min}$ (асим.)	Грана (асим.)
1	0.04	1 048 575	0.01	919 442
2	0.0268	2 097 151	0.00618	1 855 434
4	0.017956	4 194 303	0.003819	3 731 355
8	0.012031	8 388 607	0.002360	7 518 053

6.3.1 Симетрично стабло

Резултати мерења приказани су у табели 10 и табели 11, а криве скалираног убрзања на слици 11 и слици 12.

Табела 10: Слабо скалирање — симетрично стабло, Python ($p = 0.005$)

N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.538	0.008	1.000	1.000
2	1.340	0.031	0.804	1.005
4	2.092	0.080	1.029	1.016
8	4.123	0.151	1.045	1.038

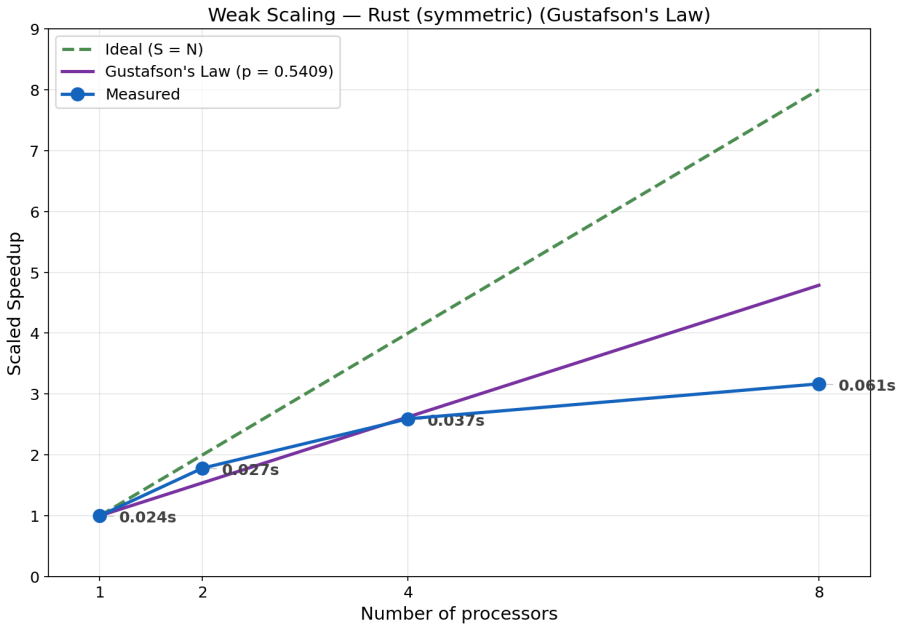


Слика 11: Скалирано убрзање при слабом скалирању — симетрично стабло, Python

Python скалирано убрзање при $N = 2$ пада испод 1 ($S_{\text{scaled}} = 0.804$) — *overhead* покретања процеса и серијализације сачињава велики удео укупног времена при малој величини проблема. При $N = 4$ и $N = 8$ рачунски обим расте довољно да *overhead* изгуби на значају, па скалирано убрзање расте изнад 1 и приближава се Густафсоној прогнози.

Табела 11: Слабо скалирање — симетрично стабло, Rust ($p = 0.541$)

N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.024	0.001	1.000	1.000
2	0.027	0.001	1.782	1.541
4	0.037	0.005	2.593	2.623
8	0.061	0.003	3.168	4.787



Слика 12: Скалирано убрзање при слабом скалирању — симетрично стабло, Rust

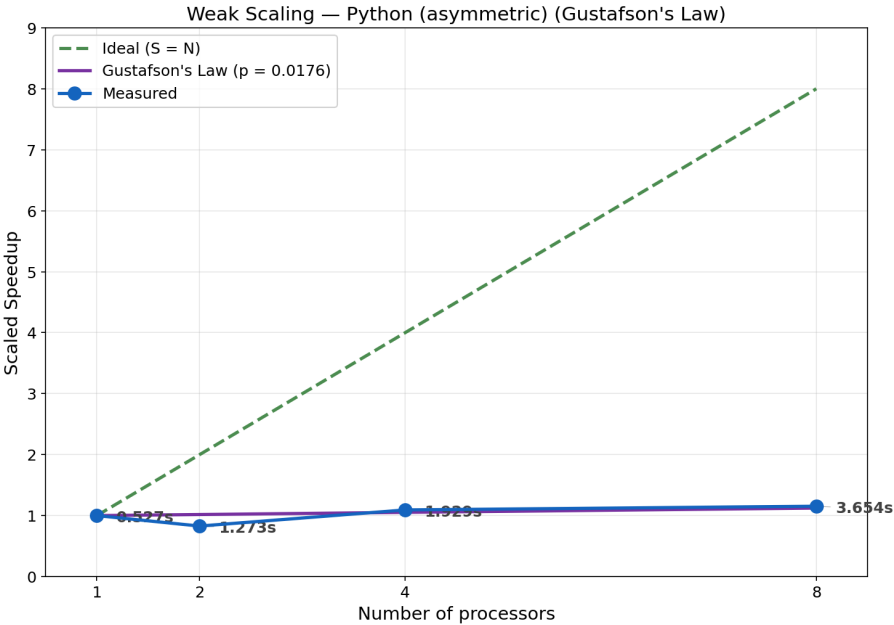
Rust скалирано убрзање монотонно расте и при $N = 4$ готово се поклапа са Густафсоновом прогнозом ($S_{\text{scaled}} = 2.593$ наспрам 2.623). При $N = 8$ измерено убрзање значајно заостаје за прогнозом ($S_{\text{scaled}} = 3.168$ наспрам 4.787) — последица *Hyper-Threading* ефекта.

6.3.2 Асиметрично стабло

Резултати мерења приказани су у табели 12 и табели 13, а криве скалираног убрзања на слици 13 и слици 14.

Табела 12: Слабо скалирање — асиметрично стабло, Python ($p = 0.018$)

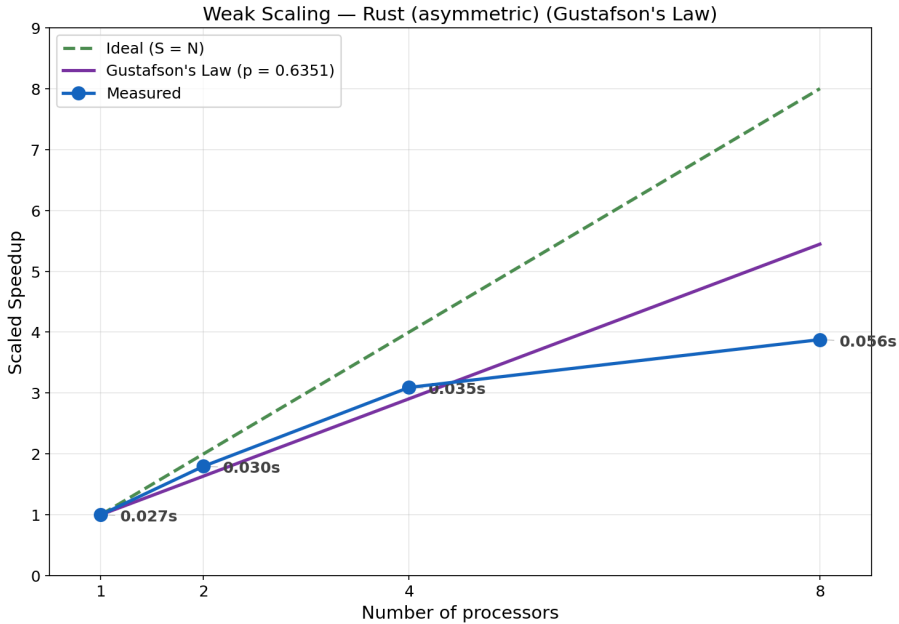
N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.527	0.007	1.000	1.000
2	1.273	0.014	0.828	1.018
4	1.929	0.091	1.093	1.053
8	3.654	0.045	1.154	1.123



Слика 13: Скалирано убрзање при слабом скалирању — асиметрично стабло, Python
Python скалирано убрзање при $N = 2$ пада испод 1 ($S_{\text{scaled}} = 0.828$), из истог разлога као у симетричном случају. При $N = 4$ и $N = 8$ расте изнад 1 и благо превазилази Густафсонову прогнозу.

Табела 13: Слабо скалирање — асиметрично стабло, Rust ($p = 0.635$)

N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.027	0.002	1.000	1.000
2	0.030	0.002	1.797	1.635
4	0.035	0.002	3.092	2.905
8	0.056	0.005	3.878	5.446



Слика 14: Скалирано убрзање при слабом скалирању — асиметрично стабло, Rust

Rust скалирано убрзање монотono расте и при $N = 2$ и $N = 4$ превазилази Густафсонову прогнозу, достижући веће вредности него у симетричном случају. При $N = 8$ заостаје за прогнозом ($S_{\text{scaled}} = 3.878$ наспрам 5.446) — последица *Hyper-Threading* ефекта.

Глава 7

Дискусија

У овом поглављу анализирају се резултати из поглавља 6 у контексту карактеристика коришћених технологија и хардверских ограничења тестне платформе. Циљ је објаснити уочене обрасце скалирања, посебно одступања од теоријских прогноза, кроз анализу механизма паралелизације у програмским језицима Python и Rust.

Сумарни преглед измерених убрзања при јаком скалирању дат је у табели 14, а при слабом скалирању у табели 15.

Табела 14: Сумарни преглед убрзања S при јаком скалирању

N	Python (сим.)	Rust (сим.)	Python (асим.)	Rust (асим.)
1	1.000	1.000	1.000	1.000
2	1.438	1.808	1.448	1.622
4	2.055	2.616	1.453	2.700
8	1.853	3.244	1.741	3.355

Табела 15: Сумарни преглед скалираног убрзања S_{scaled} при слабом скалирању

N	Python (сим.)	Rust (сим.)	Python (асим.)	Rust (асим.)
1	1.000	1.000	1.000	1.000
2	0.804	1.782	0.828	1.797
4	1.029	2.593	1.093	3.092
8	1.045	3.168	1.154	3.878

7.1 Поређење апсолутних перформанси

При секвенцијалном извршавању ($N = 1$), Rust имплементација генерише симетрично стабло приближно 39 пута брже од Python имплементације (0,204 s према 7,956 s), а асиметрично стабло приближно 34 пута брже (0,245 s према 8,215 s).

Ова разлика је директна последица природе оба језика. Python припада категорији интерпретираних програмских језика, што подразумева да се изворни код може покренути непосредно, без претходног превођења у извршну датотеку зависну од циљне платформе. У оквиру *CPython* имплементације, изворни код се прво компајлира у бајткод (енг. *bytecode*), након чега виртуелна машина тај бајткод интерпретира и извршава. Поред тога, Python користи динамичку типизацију (енг. *dynamic typing*)

— провера типова не одвија се пре покретања програма, него у току извршавања, непосредно пре обављања примитивних операција као што су аритметичке операције или приступ атрибутима објеката. Управљање меморијом у *CPython* имплементацији заснива се на бројачима референци — сваки објекат прати колико референци на њега постоји у датом тренутку, и аутоматски се ослобађа из меморије када тај број достигне нулу [21]. За разлику од интерпретираних језика, Rust је унапред компајлирани језик (енг. *ahead-of-time compiled*) — целокупан изворни код преводи се у машински извршну датотеку пре покретања програма, без потребе за присуством *runtime* окружења [19]. Генерисање фракталног стабла представља рачунски интензиван задатак (*CPU-bound*), у коме програм не чека на спољне ресурсе попут улазно-излазних операција или мрежне комуникације, већ континуирано користи процесор за извршавање рекурзивних математичких израчунавања. У таквим условима, карактеристике извршног модела програмског језика — као што су трошкови интерпретације бајткода, динамичке провере типова и управљања меморијом — директно утичу на укупно време извршавања. Из тог разлога, генерисање фракталног стабла представља погодан задатак за поређење перформанси интерпретираног и компајлираног језика.

Наведена разлика представља важан контекст при тумачењу резултата скалирања. Упркос томе што Python може постићи слично релативно убрзање у односу на сопствену секвенцијалну вредност, у апсолутном смислу остаје знатно спорији од Rust имплементације.

7.2 Јако скалирање

7.2.1 *Overhead* Python процесног управљања

Централни узрок ограниченог скалирања Python имплементације је *overhead* настао при управљању процесима, који делује на два нивоа.

На Windows платформи, Python користи *spawn* методу покретања нових процеса — при сваком позиву `Pool(processes=N)` покреће се N потпуно нових Python интерпретера. За разлику од *fork* методе доступне на Unix системима, која само копира меморијски простор постојећег процеса, *spawn* мора испочетка учитати интерпретер, увести све библиотеке и иницијализовати извршно окружење [21]. Овај процес траје мерљиво дуго и понавља се при сваком мерењу.

Поред тога, комуникација са радним процесима одвија се посредством *pickle* серијализације [21]. Задаци се пре слања пакују у бајт-ток, а резултати — *NumPy* низови са стотинама хиљада до милиона грана по подстаблу — серијализују се при враћању главном процесу. Број позива серијализације расте са N , а укупна количина серијализованих података расте са величином проблема.

Заједнички ефекат ова два фактора видљив је у конкретним мерењима: при $N = 8$, Python симетрично стабло извршава се за 4.294 s — спорије него при $N = 4$ (3.872 s). Убрзање при $N = 8$ пада испод убрзања при $N = 4$, јер при $N = 8$ систем покреће 8 нових Python интерпретера и десеријализује 8 скупова резултата, двоструко више него при $N = 4$, па трошак комуникације надмашује трошак самог рачунања.

При $N = 4$ измерено убрзање премашује Амдалову прогнозу. То произлази из чињенице да се параметар p одређује усредњавањем преко свих вредности $N > 1$. Мерења при $N = 8$, код којих *overhead* значајно нарушава скалирање, вуку процену p наниже — тиме потцењујући ефективни паралелни удео при мањим вредностима N . Последица је да Амдалова крива за $N = 4$ прогнозира мање убрзање него што мерење стварно показује.

7.2.2 Rust и *Hyper-Threading* ефекат

Rust имплементација остварује паралелизам нитима посредством библиотеке *Rayon*. За разлику од Python процеса, нити не захтевају покретање новог интерпретера нити серијализацију података — деле меморијски простор процеса и директно приступају задацима и резултатима [20]. *Overhead* трошак управљања нитима занемарљив је у поређењу са рачунским оптерећењем, па Rust убрзање прати теоријску прогнозу знатно верније него Python.

И поред тога, Rust убрзање при $N = 8$ значајно заостаје за Амдаловом прогнозом: за симетрично стабло измерено је 3.244 наспрам прогнозе 3.725. Узрок лежи у хардверском ограничењу: тестна платформа поседује 4 физичка процесорска језгра са технологијом *Hyper-Threading*, која свако физичко језгро представља као два логичка [11]. При $N = 8$ свих осам нити распоређује се на свега четири физичка језгра — две нити на истом физичком језгру деле исте извршне јединице и L1/L2 кеш меморију [23, стр. 14]. *Hyper-Threading* побољшава искоришћеност при задацима са чекањем на меморијске операције, али код *CPU-bound* задатака као што је генерисање стабла не обезбеђује линеарно повећање рачунске снаге [23, стр. 13]. Резултат је да прелазак са $N = 4$ на $N = 8$ не доноси двоструко убрзање, иако се број логичких јединица удвостручује.

7.2.3 Утицај асиметрије стабла

У Python имплементацији асиметрична подстабла директно узрокују стагнацију убрзања при $N = 4$: `pool.map` расподељује задатке статички и чека на завршетак свих процеса пре него што прикупи резултате [21]. Ако неки процес добије знатно веће подстабло од осталих, он постаје уско грло које задржава цео систем. При $N = 4$ тај ефекат доминира над добити од паралелизма, па је убрзање (1.453) готово идентично убрзању при $N = 2$ (1.448) — додатна два процеса практично не доносе напредак.

Rust имплементација овај проблем решава алгоритмом крађе посла (енг. *work-stealing*): нит која заврши свој задатак одмах преузима следећи задатак из реда нити са највише преосталог посла. Тиме се неједнаки задаци динамички расподељују међу нитима, без беспосленог чекања [20]. Последица је да Rust убрзање при $N = 8$ за асиметрично стабло (3.355) готово савршено одговара Амдаловој прогнози (3.361) — боље поклапање него у симетричном случају (3.244 према 3.725), где је крађа посла мање релевантна јер су сви задаци исте величине.

7.3 Слабо скалирање

Резултати слабог скалирања указују на два различита понашања: Python имплементација готово не скалира, а Rust остварује умерено до добро скалирање ограничено *Hyper-Threading* ефектом при $N = 8$.

Python скалирано убрзање при $N = 2$ пада испод 1 за обе конфигурације стабла (0.804 симетрично, 0.828 асиметрично). То значи да паралелна верзија са два процеса и двоструко већим проблемом извршава посао спорије него секвенцијална верзија са половициним проблемом. Разлог је *overhead* `spawn + pickle` серијализације описан у секцији 7.2.1 — при малом проблему са којим слабо скалирање почиње тај *overhead* чини велики удео укупног времена, па паралелна верзија ради спорије него секвенцијална. Процењени паралелни удео практично је нула ($p = 0.005$ симетрично, $p = 0.018$ асиметрично) — Густафсонов модел тумачи ово тако да скоро целокупно извршавање делује серијски, иако је реч о *overhead* последици, а не о суштинским серијским деловима алгоритма. При $N = 4$ и $N = 8$ рачунски део постаје довољно велик да *overhead* изгуби релативну важност, па скалирано убрзање расте изнад 1 и приближава се Густафсоновој прогнози.

Rust скалирано убрзање расте монотono за оба типа стабла, са значајним паралелним уделом ($p = 0.541$ симетрично, $p = 0.635$ асиметрично). Вредност p за асиметрично стабло је виша него за симетрично при свакој измереној вредности N — супротно очекивању на основу резултата јаког скалирања. Објашњење лежи у улози алгоритма крађе посла: при слабом скалирању, проблем расте и задаци постају неравномернији по величини, па механизам крађе посла активније расподељује посао међу нитима и остварује израженију предност него у симетричном случају где су сви задаци исте величине. Одступање од прогнозе при $N = 8$ (3.168 према 4.787 симетрично, 3.878 према 5.446 асиметрично) директна је последица *Hyper-Threading* ефекта описаног у секцији 7.2.2 — исто физичко ограничење делује и у режиму слабог скалирања.

7.4 Ограничења примењених модела

Амдалов и Густафсонов закон пружају корисну теоријску основу за квантификацију паралелног потенцијала, али почивају на претпоставкама које у пракси нису испуњене.

Оба модела претпостављају да је паралелни удео p константан и независан од броја процесорских јединица N . У стварности, *overhead* управљања процесима расте са N — при сваком додатом процесу долази до новог покретања интерпретера и нове серијализације. Ти фиксни трошкови увећавају серијски удео са сваким кораком, па модел систематски прецењује убрзање при великим вредностима N . Ефекат је нарочито изражен у слабом скалирању, где модел процењује $p \approx 0$ иако алгоритам садржи суштински паралелне делове.

Ниједан модел не узима у обзир *Hyper-Threading*: претпостављају да свако логичко језгро еквивалентно доприноси рачунској снази. Мерења показују да прелазак са $N = 4$ на $N = 8$ не даје двоструко убрзање ни у једној имплементацији, пошто четири додатна логичка језгра деле физичке извршне јединице са постојећих четири.

Додатно ограничење представља термално пригушење (енг. *throttling*) лаптоп процесора при дуготрајним оптерећењима. Тестна платформа може смањити тактну фреквенцију да би спречила прегревање, нарочито при $N = 8$ где сва логичка језгра раде на пуном оптерећењу. Циљ је да се спречи оштећење хардвера. Ово уноси систематску грешку у мерења за коју Амдалов и Густафсонов модел не пружају корекцију.

У раду је истражена ефикасност паралелизације генерисања фракталног стабла у програмским језицима Python и Rust, са циљем да се утврди у којој мери оба приступа искоришћавају расположиве процесорске ресурсе и колико верно прате теоријске прогнозе скалирања.

Резултати показују да избор механизма паралелизације одлучујуће утиче на ефикасност, независно од броја процесорских јединица. Python имплементација не остварује практичну скалабилност за *CPU-bound* задатке на оперативном систему Windows: трошак покретања интерпретера и серијализације резултата расте са N и при $N = 8$ надмашује корист паралелизма. Rust имплементација, захваљујући нитима са директним приступом меморији и алгоритму крађе посла, остварује скалирање које верно прати Амдалову прогнозу до границе хардверског ограничења — ефекта *Hyper-Threading* технологије при $N = 8$.

Одабрана стратегија паралелизације, подела стабла на независна подстабла на дубини `split_depth`, потврђена је као исправна за обе конфигурације. Кључна разлика између имплементација лежи у расподели задатака: Python их расподељује статички, па ако неки процес добије веће подстабло, остали беспослено чекају. Rust их расподељује динамички, нит која заврши свој задатак одмах преузима следећи — па неједнакост међу подстаблима не узрокује чекање.

За рачунски интензивне паралелне задатке попут генерисања фракталног стабла, приступ заснован на нитима показао се практичнијим од приступа заснованог на процесима. Вишепроцесна паралелизација може бити ефикасна тамо где је рачунање спорије у односу на трошак комуникације — за *CPU-bound* задатке попут овог, тај услов није испуњен.

Могући правци даљег истраживања укључују:

- замену `pickle` серијализације дељеном меморијом посредством `multiprocessing.shared_memory`, чиме би се елиминисао *overhead* преноса резултата у Python имплементацији;
- евалуацију на системима са `fork` методом покретања процеса (Linux), ради искључивања трошка покретања интерпретера и издавања утицаја серијализације;

-
- евалуацију на платформи са вишим бројем физичких процесорских језгара без *Hyper-Threading* технологије, ради провере Амдалових и Густафсонових прогноза при $N > 4$;

Списак слика

Слика 1	Фрактална структура гранања стабала у природи [4].	3
Слика 2	Пример рачунарски генерисаног бинарног фракталног стабла [8].	4
Слика 3	Симетрично (лево) и асиметрично (десно) фрактално стабло.	5
Слика 4	Распоред меморијског простора процеса [11, стр. 107].	7
Слика 5	Амдалов закон (горе) — убрзање асимптотски тежи $1/f$ [16]; Густафсонов закон (доле) — убрзање расте линеарно са бројем процесора [17].	13
Слика 6	Убрзање у зависности од <code>split_depth</code> за асиметрично стабло при $N = 8$. Испрекидана линија означава хеуристичку вредност $d = 5$	17
Слика 7	Убрзање при јаком скалирању — симетрично стабло, Python	24
Слика 8	Убрзање при јаком скалирању — симетрично стабло, Rust	25
Слика 9	Убрзање при јаком скалирању — асиметрично стабло, Python	26
Слика 10	Убрзање при јаком скалирању — асиметрично стабло, Rust	27
Слика 11	Скалирано убрзање при слабом скалирању — симетрично стабло, Python	28
Слика 12	Скалирано убрзање при слабом скалирању — симетрично стабло, Rust	29
Слика 13	Скалирано убрзање при слабом скалирању — асиметрично стабло, Python	30
Слика 14	Скалирано убрзање при слабом скалирању — асиметрично стабло, Rust	31

Списак листинга

Листинг 1	Мемоизована функција за бројање грана асиметричног подстабла	18
Листинг 2	Рекурзивна функција за бројање грана асиметричног подстабла — Rust	20

Списак табела

Табела 1	Хардверска конфигурација тестног система	21
Табела 2	Верзије коришћених компоненти	21
Табела 3	Параметри фракталног стабла и конфигурација мерења	22
Табела 4	Вредности <code>split_depth</code> по конфигурацији; број задатака износи $2^{\text{split_depth}}$	23
Табела 5	Јако скалирање — симетрично стабло, Python ($p = 0.607$)	23
Табела 6	Јако скалирање — симетрично стабло, Rust ($p = 0.836$)	24
Табела 7	Јако скалирање — асиметрично стабло, Python ($p = 0.507$)	25
Табела 8	Јако скалирање — асиметрично стабло, Rust ($p = 0.803$)	26
Табела 9	Вредности $l_{\min}$ и број грана по конфигурацији за слабо скалирање	28
Табела 10	Слабо скалирање — симетрично стабло, Python ($p = 0.005$)	28
Табела 11	Слабо скалирање — симетрично стабло, Rust ($p = 0.541$)	29
Табела 12	Слабо скалирање — асиметрично стабло, Python ($p = 0.018$)	30
Табела 13	Слабо скалирање — асиметрично стабло, Rust ($p = 0.635$)	30
Табела 14	Сумарни преглед убрзања S при јаком скалирању	33
Табела 15	Сумарни преглед скалираног убрзања S_{scaled} при слабом скалирању . . .	33

Списак коришћених скраћеница

Скраћеница	Опис
GIL	Global Interpreter Lock (глобална брава интерпретера)
IPC	Inter-Process Communication (међупроцесна комуникација)
IQR	Interquartile Range (интерквартилни распон)
PCB	Process Control Block (блок контроле процеса)

Списак коришћених појмова

Појам	Објашњење
Бинарно стабло	Хијерархијска структура у којој је сваки унутрашњи чвор повезан са тачно два потомка — левим и десним.
Глобална брава интерпретера (GIL)	Механизам у <i>CPython</i> имплементацији којим се ограничава извршавање на само једну нит истовремено.
Јако скалирање	Мерење убрзања при фиксној величини проблема и растућем броју процесорских јединица.
Крађа посла	Стратегија динамичке расподеле оптерећења у којој нит без сопственог посла преузима задатке из реда друге, заузете нити.
Нит	Основна јединица извршавања унутар процеса; меморијски простор се дели међу нитима истог процеса.
Процес	Програм у извршавању са сопственим меморијским простором, изолован од осталих процеса.
Рекурзија	Техника решавања проблема при којој се функција рекурзивно позива са умањеним потпроблемом, све до достизања основног случаја.
Слабо скалирање	Мерење убрзања при коме се величина проблема повећава сразмерно броју процесорских јединица.
Термално пригушење	Аутоматско снижавање радне фреквенције процесора ради заштите од прегревања.
Фрактал	Геометријска фигура која се може поделити на делове при чему је сваки од тих делова умањена копија целе фигуре.

Биографија

Ана Попарић рођена је 28. децембра 2002. године у Аранђеловцу. Основну школу „Илија Гарашанин“ у Аранђеловцу завршила је 2017. године као носилац Вукове дипломе. Исте године уписује Гимназију „Милош Савковић“ у Аранђеловцу, коју завршава 2021. године са одличним успехом. Након завршене средње школе, уписује основне академске студије на Факултету техничких наука у Новом Саду, на одсеку Софтверско инжењерство и информационе технологије. Положила је све испите предвиђене наставним планом и програмом.

Литература

- [1] „Фрактал“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на <https://sr.wikipedia.org/wiki/%D0%A4%D1%80%D0%B0%D0%BA%D1%82%D0%B0%D0%BB>
- [2] B. Mellor, „Fractals: Self-Similarity and Fractal Dimension“. Приступљено: 15. Јуни 2026. [На Интернету]. Доступно на <https://blakemellor.lmu.build/courses/Symmetry/FractalDimension-Spring2013.pdf>
- [3] H. Gupta, „Fractals in Nature: A Mathematical Perspective“, *International Journal for Research Publication and Seminars*, том 16, изд. 1, стр. 581–584, 2025, doi: [10.36676/jrps.v16.i1.322](https://doi.org/10.36676/jrps.v16.i1.322).
- [4] M. Sabroe, „Fractals Tree on Knudshoved Odde“. Приступљено: 15. Јуни 2026. [На Интернету]. Доступно на https://commons.wikimedia.org/wiki/File:Fractals_Tree_on_Knudshoved_Odde_-_panoramio.jpg
- [5] „Fractals and Recursive Graphics“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1208/lectures/fractals/Lecture10_Slides.pdf
- [6] „Binary tree“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на https://en.wikipedia.org/wiki/Binary_tree
- [7] „Fractal Trees – Definitions“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на <https://www.math.union.edu/research/fractaltrees/FractalTreesDefs.html>
- [8] „Fractal Trees – Definitions“. Приступљено: 15. Јуни 2026. [На Интернету]. Доступно на <https://www.math.union.edu/research/fractaltrees/FractalTreesDefs.html>
- [9] M. Anderson и others, „Fractal Trees“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на <https://www.ithaca.edu/sites/default/files/2021-11/fractal-trees-poster.pdf>
- [10] M. F. Barnsley, *Fractals Everywhere*, 2nd изд. Academic Press, 1993.
- [11] A. Silberschatz, P. B. Galvin, и G. Gagne, *Operating System Concepts*, 10th изд. Wiley, 2018.
- [12] J. Dinan, S. Olivier, G. Sabin, J. Prins, P. Sadayappan, и C.-W. Tseng, „Dynamic Load Balancing of Unbalanced Computations Using Message Passing“, у *Workshop on Performance Modeling, Evaluation, and Optimization of Parallel and Distributed*

-
- Systems (PMEO-PDS)*, IEEE, 2007. Приступљено: 17. Јуни 2026. [На Интернету].
Доступно на <https://www.cs.unc.edu/~olivier/pmeo07.pdf>
- [13] E. Khomenko и others, „Mancha3D Code: Multi-Purpose Advanced Non-Ideal MHD Code for High Resolution Simulations in Astrophysics“, *arXiv preprint arXiv:2312.04179*, 2024, Приступљено: 22. Јуни 2026. [На Интернету]. Доступно на <https://arxiv.org/pdf/2312.04179>
- [14] J. L. Gustafson, „Reevaluating Amdahl's Law“, *Communications of the ACM*, том 31, изд. 5, стр. 532–533, 1988.
- [15] L. Yavits, A. Morad, и R. Ginosar, „The Effect of Communication and Synchronization on Amdahl's Law in Multicore Systems“. Приступљено: 18. Јуни 2026. [На Интернету]. Доступно на <https://arxiv.org/pdf/1306.3302>
- [16] Daniels220, „Amdahl's Law“. Приступљено: 18. Јуни 2026. [На Интернету]. Доступно на <https://commons.wikimedia.org/wiki/File:AmdahlsLaw.svg>
- [17] „Gustafson's Law“. Приступљено: 18. Јуни 2026. [На Интернету]. Доступно на <https://commons.wikimedia.org/wiki/File:Gustafson.png>
- [18] A. Ajitsaria, „What Is the Python Global Interpreter Lock (GIL)?“. Приступљено: 22. Јуни 2026. [На Интернету]. Доступно на <https://realpython.com/python-gil/>
- [19] S. Klabnik и C. Nichols, „The Rust Programming Language“. Приступљено: 22. Јуни 2026. [На Интернету]. Доступно на <https://doc.rust-lang.org/book/>
- [20] N. Matsakis и J. Stone, „Rayon: A data parallelism library for Rust“. Приступљено: 19. Јуни 2026. [На Интернету]. Доступно на <https://github.com/rayon-rs/rayon>
- [21] Python Software Foundation, „Python 3 Documentation“. [На Интернету]. Доступно на <https://docs.python.org/>
- [22] Intel Corporation, „Information about Temperature for Intel® Processors“. Приступљено: 25. Јуни 2026. [На Интернету]. Доступно на <https://www.intel.com/content/www/us/en/support/articles/000005597/processors.html>
- [23] Intel Corporation, „Intel® Hyper-Threading Technology Technical User's Guide“. Јануар 2003.